

Question 1: Bank Management System:

Scenario:

XYZ Bank wants a simplified account management system. The bank offers two types of accounts: **Savings Account** and **Current Account**. Every account has an account number, account holder name, and balance. The bank has specific rules:

- Savings accounts earn monthly interest and cannot go below a minimum balance of Rs. 1000.
- Current accounts allow overdrafts up to a fixed limit (e.g., Rs. 5000) but earn no interest.
- Account balances must never be modified directly from outside the class — all changes must go through controlled methods (deposit/withdraw).
- All account types must support a common way to calculate and display monthly interest/charges, even though the calculation differs.

Tasks:

- a) Define a class `BankAccount` with common properties (account number, holder name, balance) as **encapsulated (private/protected)** fields with getter methods, and a method `applyMonthlyUpdate()` with a basic default implementation.
- b) Create classes `SavingsAccount` and `CurrentAccount` that **inherit** from `BankAccount` and **override** `applyMonthlyUpdate()` differently — interest addition for Savings, overdraft fee/no charge for Current.
- c) Implement `deposit(amount: number)` and `withdraw(amount: number)` methods in the base class using **encapsulation**, ensuring the Savings account enforces the minimum balance rule and the Current account enforces the overdraft limit (override where necessary).
- d) Write a function `processMonthEnd(accounts: BankAccount[])` that loops through an array of mixed account **objects** and calls `applyMonthlyUpdate()` on each, demonstrating **polymorphism**.
- e) In your main program, create at least one `SavingsAccount` object and one `CurrentAccount` object, perform a deposit and a withdrawal on each, and call `processMonthEnd()`.

Question 2: Hospital Management System (25 marks)

Scenario:

A hospital wants software to manage its staff. All staff members share common details: staff ID, name, and monthly base salary. However:

- **Doctors** receive a consultation allowance per patient seen and can `diagnose()` a patient.
- **Nurses** receive a shift allowance based on number of shifts worked and can `assist()` in a procedure.
- Salary calculation differs for each role, but every staff member must be able to report their **total monthly salary** through a common method.
- Sensitive data such as base salary should not be directly editable from outside the class; it should only be updated through a controlled method (e.g., `updateSalary()`) that validates the new value is positive.

Tasks:

- a) Design a class `Staff` with **encapsulated** fields (`staffId`, `name`, `baseSalary`), a private/protected access modifier for salary, a getter, an `updateSalary(newSalary: number)` method with validation, and a method `calculateTotalSalary(): number` with a basic default implementation.
- b) Create `Doctor` and `Nurse` classes that **inherit** from `Staff`, each adding its own extra property (`consultationsCount` / `shiftsWorked`) and **overriding** `calculateTotalSalary()` with its own formula.
- c) Add a method specific to each subclass — `diagnose(patientName: string)` for `Doctor` and `assist(procedureName: string)` for `Nurse` — showing that subclasses can extend behaviour beyond the parent class.
- d) Write a function `generatePayroll(staffList: Staff[])` that iterates over a mixed array of `Doctor` and `Nurse` **objects** and prints each staff member's name and total salary by calling `calculateTotalSalary()`, demonstrating **polymorphism**.
- e) In your main program, instantiate at least one `Doctor` and one `Nurse` object, call their unique methods, attempt an invalid `updateSalary()` call (e.g., negative value) to show encapsulation protecting the data, and run `generatePayroll()`.